GenAI / LLM Production Interview Questions

Scenario-based interview question bank generated from the uploaded handwritten notes. Focus: production GenAI, RAG, agents, security, reliability, observability, and system design.

1. Authentication, Authorization & API Security

1. Explain Access Token vs Refresh Token. What is the purpose of each, where should each token be stored, and when should each expire?

2. A refresh token is stolen. What is your production mitigation strategy? Explain rotation, revocation, reuse detection, and session invalidation.

3. How would you design OAuth2/JWT authentication for a FastAPI-based GenAI application?

4. What is the difference between authentication and authorization? Give a production example in an AI agent system.

5. How would you securely manage API keys and secrets across local development, staging, and production?

6. How do you prevent an LLM agent from calling tools that the authenticated user is not authorized to use?

2. LLM Security & Jailbreak Protection

7. A user successfully jailbreaks your LLM in production. What real-time detection and mitigation strategy would you implement?

8. How would you detect prompt injection in an LLM application? What signals would you monitor?

9. How do you defend an agentic AI system against indirect prompt injection coming from retrieved documents or web pages?

10. Where should guardrails be placed: before the LLM, after the LLM, around tools, or at multiple layers? Explain why.

11. How would you handle unsafe or policy-violating model output without breaking the user experience?

12. How would you test an LLM application against jailbreaks before production deployment?

3. Redis, Caching & Reliability

13. Your Redis cache crashes at 2 AM and the entire application goes down. Redis was only a cache—why did everything die?

14. How would you design an application so that a Redis outage does not cause a complete production outage?

15. Explain cache-aside, write-through, and write-behind caching. When would you use each?

16. How would you prevent a cache stampede when thousands of requests miss the cache at the same time?

17. How do TTL, eviction policy, cache invalidation, and stale data affect an LLM/RAG application?

18. How would you implement Redis health checks, timeouts, circuit breakers, retries, and graceful fallback?

4. Logging, Monitoring & Distributed Tracing

19. How do you implement production logging and tracing for an LLM application?

20. What information should be captured in an LLM trace without exposing sensitive user data or secrets?

21. How would you trace one user request across API → retriever → vector database → LLM → tool call → response?

22. What are logs, metrics, and traces? Give a production example for each in a RAG system.

23. Which LLM-specific metrics would you monitor: latency, token usage, cost, retrieval quality, hallucination rate, or something else?

24. How would you debug a sudden increase in LLM latency using distributed traces?

5. CI/CD & Production Deployment

25. Your CI/CD pipeline is completely green, but five minutes after deployment production starts returning 500 errors. What did the pipeline fail to test?

26. What tests should be included in a production GenAI CI/CD pipeline?

27. How would you design smoke tests, integration tests, regression tests, and LLM evaluation tests for deployment?

28. Explain blue-green deployment and canary deployment. How would you use them for an LLM service?

29. How would you automatically roll back a deployment when error rate or latency crosses a threshold?

30. How would you test production configuration, secrets, model versions, prompts, and external API dependencies before release?

6. RAG Debugging & Retrieval

31. Your RAG retrieves the correct policy, but the LLM gives the opposite answer. What failed?

32. How would you debug a RAG pipeline step by step when the final answer is wrong?

33. What is the difference between retrieval failure, context construction failure, and generation failure?

34. How do chunk size, chunk overlap, metadata, embedding model, top-k, and similarity threshold affect RAG quality?

35. When would you use hybrid search instead of pure vector search?

36. Why would you add a cross-encoder reranker after initial vector retrieval?

37. How would you handle conflicting documents or outdated policies in a RAG system?

38. How do you prevent the LLM from answering from its own knowledge when the required answer is not supported by retrieved context?

7. Vector Databases & Large-Scale Retrieval

39. Your vector database contains 10 million documents. If a user sends a query, how can the system find the top similar documents without comparing the query vector with all 10 million vectors?

40. Explain approximate nearest-neighbor (ANN) search and why it is used at scale.

41. Compare HNSW, IVF, and quantization at a high level. What trade-offs do they introduce?

42. How would metadata filtering improve retrieval performance and relevance in a large vector database?

43. How would you design a multi-tenant vector search system where each customer must only retrieve its own documents?

44. How would you reduce vector-search latency when the database grows from 1 million to 100 million vectors?

45. How do you choose top-k for retrieval, and why can an unnecessarily large k hurt the final answer?

8. Agentic AI & Tool Calling

46. Two agents use the same LLM and tools, but one performs significantly better. There is no obvious prompt difference. What hidden choices could explain the gap?

47. What hidden variables can affect agent performance: system prompt, tool descriptions, tool ordering, temperature, model version, context, memory, retries, or routing?

48. How would you design an agent so it does not enter an infinite tool-calling loop?

49. How do you validate tool inputs and outputs before allowing an agent to continue?

50. When should an agent ask for human approval instead of executing a tool automatically?

51. How would you evaluate an agent's tool selection and multi-step reasoning without relying only on the final answer?

52. How would you make an agent deterministic and reproducible enough for production debugging?

9. RAG Evaluation & Observability

53. Your RAG is live. How do you actually know it is working?

54. What offline evaluation dataset would you create before launching a RAG application?

55. Explain retrieval metrics such as Recall@K, Precision@K, MRR, and NDCG in simple terms.

56. How would you evaluate answer faithfulness, answer relevance, and context relevance?

57. How would you use Ragas or a similar evaluation framework in CI/CD?

58. How do you detect RAG quality degradation after changing the embedding model, chunking strategy, prompt, or LLM?

59. What production feedback signals would you collect to continuously improve a RAG system?

10. Document Ingestion & PDF Processing

60. You upload a 500-page PDF to an AI chatbot. How would you process, chunk, embed, index, and retrieve the content efficiently?

61. A PDF contains tables, images, scanned pages, and text. How would you build a robust ingestion pipeline?

62. When should you use OCR, and how would you validate OCR quality?

63. How would you preserve page numbers, document IDs, headings, tables, and other metadata during chunking?

64. How would you update or delete documents from a vector database without rebuilding the entire index?

65. How would you handle duplicate documents and multiple versions of the same document?

11. Caching, Cost & Performance

66. How would you cache repeated LLM requests safely? What should be part of the cache key?

67. What is semantic caching, and when is it safer or more useful than exact-response caching?

68. How would you reduce LLM cost while maintaining answer quality?

69. How would you reduce end-to-end RAG latency when retrieval and generation are both expensive?

70. How would you decide whether to cache embeddings, retrieval results, LLM responses, or all three?

71. How would you prevent stale cached answers when the underlying knowledge base changes?

12. Production System Design Scenarios

72. Design a production-grade RAG API using FastAPI, Redis, a vector database, an LLM provider, and PostgreSQL.

73. Design a highly available GenAI application where the LLM provider can become unavailable.

74. Design a multi-tenant enterprise RAG system with authentication, authorization, document isolation, observability, and audit logs.

75. Design an agentic customer-support system that can retrieve policies, query order data, and create support tickets with human approval.

76. How would you design rate limiting for an LLM API when model calls are expensive and have provider-specific limits?

77. How would you handle model-provider failover when the primary LLM API becomes unavailable?

78. How would you version prompts, models, embeddings, retrieval configuration, and evaluation datasets?

Total: 78 interview questions — designed for practical production-level preparation.